CAPSTONE PROJECT 01

ShopSmart: E-commerce Recommendation and Shopping Assistant

Sakura Home, an online home goods retailer in India and Japan with 2,000 products and 30,000 customers.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

1. The customer's problem

Customers struggle to find relevant products; the home page shows the same best sellers to everyone and search only matches exact words. Sakura Home wants personalised recommendations and a shopping assistant that answers product questions accurately, without inventing prices or stock.

2. Users

- Shoppers browsing on web and mobile
- Merchandising team who curate featured products
- Customer support team who answer product questions

3. User stories

- As a shopper, I see products recommended for me based on what I viewed and bought.
- As a shopper, I ask "a quiet kettle for a small Tokyo apartment under 6,000 JPY" and get matching products with reasons.
- As a shopper, I see "customers also bought" on a product page.
- As a merchandiser, I see which recommendations are clicked and bought.
- As support, I trust that the assistant never invents prices or stock.

4. Requirements

Must have

- Product catalogue API with search (keyword + semantic hybrid)
- Recommendations: popularity baseline, item-to-item co-purchase, and content-based with embeddings
- Shopping assistant (RAG over product descriptions and FAQs) that cites product ids and reads price and stock from the database, never from the model
- React storefront: home recommendations, search, product page with "also bought", assistant panel

- Offline evaluation: precision@k and hit rate@k on held-out purchases; golden set for the assistant

Should have

- Cold-start strategy for new users
- Click and add-to-cart events logged for a simple dashboard
- Japanese and English product names

Could have (stretch)

- A/B test design for two recommenders
- Re-ranking by margin or stock
- Basket completion suggestions

5. Data

Starter notebook generates products (id, name, category, price INR and JPY, stock, description, tags) and 60,000 interaction events (view, cart, purchase) for 3,000 customers.

6. Suggested AI approach

Embeddings for product descriptions; hybrid search with RRF (Day 23); item-item co-occurrence; grounded assistant with tool calls for price and stock (Days 22, 25).

7. Suggested architecture

```
React UI -> FastAPI back end -> database (SQL / MongoDB)
              |-> AI service (retrieval, model, tools, guardrails)
              |-> logs and metrics -> dashboard / weekly report
CI/CD: tests + quality gate -> Docker images
       -> Azure (Container Apps or App Service)
```

8. How to do it: day by day

Day 36 (Mon Dec 14): Kick-off and design

- Quiz on Day 35
- Teams of 3 to 4 formed; project chosen
- Problem statement, users, success metrics, assumptions log
- Architecture diagram and data model

Day 37 (Tue Dec 15): Core build

- Stand-up (15 min)
- Data loaded; back end API with main endpoints
- AI core working end to end (retrieval, recommendation, agent or pipeline)
- Golden set draft of 20 items

Day 38 (Wed Dec 16): Complete features and quality

- Stand-up

- Front end connected to the API
- Guardrails and error handling
- Evaluation run and first error analysis

Day 39 (Thu Dec 17): Harden, deploy, observe

- Stand-up
- Docker images; deploy to Azure or a documented local deployment
- Structured logs and a monitoring view
- Red-team and slice checks; fix top issues

Day 40 (Fri Dec 18): Demo day

- Final rehearsal (morning)
- Demo: 10 minutes + 5 minutes Q&A; per team
- Evaluator scoring with the rubric
- Individual reflection and peer feedback

9. Project-specific acceptance checks

Evaluators run these during or after the demo. They count towards Core functionality and AI quality.

- Recommendations beat the popularity baseline on hit rate@10
- Assistant answers include product ids; prices come from the database
- Guardrail blocks requests for other customers' orders
- Latency p95 under 3 s for the assistant

10. Golden set: starter items

Extend these to at least 20 items (30 for PolicyPal) with expected answers, sources and tags.

- "Show me a kettle under 3,000 INR" must only return kettles priced under 3,000 INR
- "Is the Aoi lamp in stock?" must match database stock
- "What is the cheapest product?" must not invent a product

11. How you will be evaluated

Criterion	Points	For this project, evaluators look for
Problem understanding and scoping	10	Clear problem statement, users, success metrics, assumptions log, agreed MVP scope (Days 15, 20).
Architecture and design	10	Sensible architecture diagram, layered code, right tool choices with reasons, data model (Days 4, 6, 7, 22, 33).
Core functionality	20	All must-have features work end to end in the demo; project checklist items delivered.
AI and data quality	15	Golden set of at least 20 items, measured metrics (retrieval, faithfulness or recommendation quality), error analysis (Days 23, 29, 30).
Engineering quality	10	Readable code, tests (API, pipeline, prompts), Git history with pull requests, CI pipeline green (Days 13, 17, 18).
Security, guardrails and responsible AI	10	Input and output guardrails, secrets handling, least privilege, a red-team check, fairness or language slice check (Days 8, 28).
Deployment and operations	10	Containerised, deployable (Azure or documented), structured logs, a dashboard or report, one drift or quality check (Days 19, 31 to 35).
Demo and communication	10	Answer-first readout, live demo, honest limits, confident Q&A;, clear handover README (Days 15, 20, 24).
Teamwork and process	5	Daily stand-ups, decision log, fair contribution, kind and specific feedback (Days 15, 24, 31).

12. Common pitfalls

- Letting the model state prices from memory
- Evaluating on the same purchases used to build recommendations (data leakage)
- Skipping the popularity baseline

13. Getting started

- Open Starter_Notebooks/Capstone_01_ShopSmart_Starter.ipynb in Colab or VS Code to generate the data and see a working baseline.
- Use App_Build_Prompts/Capstone_01_ShopSmart_Prompts.md with your AI coding assistant to scaffold the project.
- Write your problem statement and assumptions log before writing code.

Ganbatte (頑張って): do your best